Arabian Journal for Science and Engineering (2025) 50:1191–1207
https://doi.org/10.1007/s13369-024-09140-0

RESEARCH ARTICLE-COMPUTER ENGINEERING AND COMPUTER SCIENCE

Swift Detection of XSS Attacks: Enhancing XSS Attack Detection by Leveraging Hybrid Semantic Embeddings and AI Techniques

Rezan Bakır[1] · **Halit Bakır**[1]

Received: 8 August 2023 / Accepted: 30 April 2024 / Published online: 3 June 2024

Abstract
Cross-Site Scripting (XSS) attacks continue to be a significant threat to web application security, necessitating robust detection mechanisms to safeguard user data and ensure system integrity. In this study, we present a novel approach for detecting XSS attacks that harnesses the combined capabilities of the Universal Sentence Encoder (USE) and Word2Vec embeddings as a feature extractor, aiming to enhance the performance of machine learning and deep learning techniques. By leveraging the semantic understanding of sentences offered by USE and the word-level representations from Word2Vec, we obtain a comprehensive feature representation for XSS attack payloads. Our proposed approach aims to capture both fine-grained word meanings and broader sentence contexts, leading to enhanced feature extraction and improved model performance. We conducted extensive experiments utilizing machine learning and deep learning architectures to evaluate the effectiveness of our approach. The obtained results demonstrate that our combined embeddings approach outperforms traditional methods, achieving superior accuracy, precision, recall, ROC, and F1-score in detecting XSS attacks. This study not only advances XSS attack detection but also highlights the potential of state-of-the-art natural language processing techniques in web security applications. Our findings offer valuable insights for the development of more robust and effective security measures against XSS attacks.

Keywords Universal Sentence Encoder · Word2vec · Word embedding · XSS attack · Machine learning · Deep learning

1 Introduction

Cross-Site Scripting (XSS) attacks pose a significant threat to the security of web applications, making it imperative for developers and security practitioners to understand the nature and implications of these attacks [1]. XSS attacks are a type of injection attack where malicious actors exploit vulnerabilities in a web application to inject malicious code into trusted websites. This code is then executed by unsuspecting users, leading to unauthorized access, data theft, or other malicious activities [2].

The impact of XSS attacks can be severe, ranging from compromising sensitive user information to spreading malware and phishing attempts [3]. These attacks exploit the trust between a user and a website, taking advantage of the dynamic nature of web content. Traditional security measures such as input validation and output encoding are often insufficient to prevent XSS attacks, as the sophistication and variety of attack vectors continue to evolve [4].

Researchers in the field of web application security have been investigating advanced techniques to evade detection and obfuscate malicious code. This includes the use of various evasion techniques, encoding methods, and payload obfuscation to bypass detection mechanisms employed by security systems [5].

Addressing XSS attacks requires a comprehensive approach that combines robust security practices, regular security audits, and advanced detection mechanisms. In recent years, the integration of deep learning and machine

Rezan Bakır or Razan Ghanem: the author's name is written in two different ways because of having dual citizenship. Halit Bakır: Khaled Bakour: the author's name is written in two different ways because of having dual citizenship.

✉ Rezan Bakır
rezan.bakir@sivas.edu.tr

Halit Bakır
halit.bakir@sivas.edu.tr

[1] Department of Computer Engineering, Sivas University of Science and Technology, Sivas, Turkey

learning techniques has shown promising results in enhancing the detection capabilities of XSS attacks [6–10]. These techniques leverage the power of artificial intelligence to analyze web content, identify patterns, and detect potential malicious code.

One key aspect of effective XSS attack detection lies in the ability to quickly analyze and identify malicious scripts within web content. This is where the Universal Sentence Encoder takes center stage. The Universal Sentence Encoder is a powerful tool that enables the transformation of textual data into high-dimensional vectors, capturing semantic similarities and meaning [11]. By utilizing the Universal Sentence Encoder as a feature extractor, XSS detection systems can rapidly process and analyze web content, and efficiently identifying potential XSS attacks. In this article, we will explore the utilization of the Universal Sentence Encoder as a feature extractor in conjunction with artificial intelligence techniques to enhance the speed and accuracy of XSS attack detection. We will delve into the intricacies of XSS attacks, discuss the capabilities of the Universal Sentence Encoder, and examine how it can be integrated into existing detection systems. By combining the strengths of deep learning, machine learning, and the Universal Sentence Encoder with Word2vec models, we aim to contribute to the advancement of XSS attack detection and bolster web application security.

The rapid detection of XSS attacks is essential for effectively mitigating their impact on web applications and user data. Speed is crucial in thwarting active attacks and minimizing potential damage. Real-time detection allows for immediate response and mitigation actions, such as sanitizing user inputs and patching security loopholes. Furthermore, given the dynamic nature of web applications, real-time monitoring is necessary to identify and address threats promptly. Swift detection reduces the attack surface and mitigates the risk of data exfiltration or unauthorized actions. It also limits attackers' ability to exploit vulnerabilities, enabling proactive responses and strengthening overall security. The motivation behind this research stems from the pressing need to improve the detection of XSS attacks in web applications. As XSS attacks continue to evolve and grow in complexity, it is crucial to stay one step ahead of malicious actors and safeguard user data and online experiences.

Traditional methods of XSS detection often rely on signature-based approaches or rule-based heuristics, which may be limited in their effectiveness and struggle to keep up with emerging attack techniques. Furthermore, these methods can be resource-intensive and prone to false positives and false negatives, leading to inefficient security practices and potential vulnerabilities. Furthermore, the traditional methods for XSS detection often struggle to effectively capture the nuanced and context-dependent nature of XSS payloads. To overcome these limitations, we sought to explore a novel approach by combining the powerful semantic understanding of sentences offered by the Universal Sentence Encoder with the word-level representations provided by Word2Vec. This unique fusion of embeddings aimed to capture both fine-grained word meanings and broader sentence contexts, thus enhancing the feature extraction process for XSS attack payloads. By leveraging this combined feature representation, we aspired to design a more robust and accurate XSS attack detection system capable of generalizing well to diverse attack scenarios and effectively mitigating potential threats. Moreover, there is a growing interest in leveraging artificial intelligence (AI) techniques and deep learning algorithms to bolster the detection capabilities of XSS attacks. By training the proposed AI models and incorporating the Universal Sentence Encoder with Word2vec model, we aim to improve the accuracy and efficiency of XSS attack detection. The Universal Sentence Encoder offers a unique advantage in the realm of XSS detection. Its ability to transform text into meaningful, high-dimensional vectors captures semantic information and similarities between sentences. This allows us to go beyond surface-level analysis and delve into the underlying intent and context of web content. By harnessing the power of the Universal Sentence Encoder, we can extract rich features from text, enabling more effective identification of potential XSS attacks. Moreover, by combining the strengths of Universal Sentence Encoder and Word2Vec, we can potentially achieve a more comprehensive representation of our text data, incorporating both word-level and sentence-level semantics. Additionally, underscoring the criticality of swift detection in thwarting XSS attacks, we aim to harness the heightened capabilities facilitated by the Universal Sentence Encoder. Our goal is to develop a more efficient and agile XSS detection system by leveraging these speed enhancements. Through this research, we aim to contribute to the advancement of XSS attack detection techniques by combining the capabilities of the Universal Sentence Encoder and Word2vec with artificial intelligence methods.

The contribution of the paper can be abstracted as follow:

Introduction of the Universal Sentence Encoder (USE) as a novel feature extractor for XSS attack detection, enabling the capture of intricate semantic relationships and contextual information from web content. This advancement surpasses the limitations of traditional feature extraction methods and enhances the accuracy of XSS attack identification.
Comprehensive comparative analysis between the Universal Sentence Encoder and the Word2vec feature extraction methods, providing insights into their respective strengths and weaknesses in the context of XSS attack detection. This analysis serves as a valuable reference for researchers and practitioners in the field of web application security.

Evaluation of various machine learning classifiers and deep learning architectures in conjunction with the proposed feature extraction model, incorporating an analysis of processing times associated with the Universal Sentence Encoder and Word2vec approaches. This comprehensive evaluation aims to identify the most effective models for XSS attack detection while emphasizing the importance of minimal latency in real-world deployment. By establishing best practices for model selection, this research contributes significantly to enhancing the efficiency and effectiveness of XSS detection systems.

The remaining sections of the paper are structured as follows: Sect. 2 provides an overview of existing techniques employed to detect XSS attack. Section 3 outlines the methodologies employed in this study. It briefly discusses the models utilized for the comparative analysis, while delving into a comprehensive description of the proposed model. Section 4 presents the experimental results obtained from our study. Section 5 critically examines and discusses the outcomes obtained in the preceding sections. Finally, Sect. 6 encompasses the conclusion, summarizing the key findings, limitations, and the proposed future studies.

2 Related Works

The field of XSS attack detection has witnessed several advancements in recent years. Researchers have explored various methods and techniques to effectively identify and mitigate XSS attacks. Traditional approaches often rely on rule-based or signature-based methods to detect known attack patterns. However, these methods may struggle to keep up with evolving attack vectors and may result in false positives or false negatives. To overcome these limitations, researchers have turned to machine learning and deep learning techniques. These approaches leverage the power of algorithms and models to learn from large amounts of data and detect patterns indicative of XSS attacks.

According to [12] review paper, research studies focused on the application of deep learning (DL) and machine learning (ML) techniques in the context of XSS attack detection can be classified into six distinct categories: Client-side, Server-side, Hybrid client–server, Internet-of-Things (IoT), Mobile, and other miscellaneous categories.

For example, the research paper [13] exemplifies a client-side detection approach. In the study, a client-side solution called Noxes was introduced to prevent cross-site scripting attacks. Operating as a web proxy, Noxes employs both manually and automatically defined rules. Requiring minimal user engagement and modification, Noxes effectively safeguards against information leakage from the user's environment. Another study [14], proposes an approach based on the Knuth–Morris–Pratt (KMP) string matching algorithm to detect malicious code and mitigate potential threats.

In the paper [15], the researchers recommended a Bayesian network strategy for detecting XSS attacks, incorporating domain expertise and threat information. According to the authors, the method prioritizes nodes based on their influences on the output node, enhancing end-user comprehension. The effectiveness of the method's quick retaliation capabilities against XSS attacks was validated through experiments on a real-world dataset. Additionally, the study presented in [16] proposed a fuzzing-based approach that utilizes machine learning and deep learning algorithms for XSS attack detection. This approach not only increased the confidence coefficient of malicious samples but also generated adversarial attack examples using Soft Q-learning. Furthermore, the [16] study proposed a fuzzing-based approach to detect cross-site scripting (XSS) attacks using machine learning and deep learning algorithms. The approach improved the confidence coefficient of malicious samples and generated adversarial attack examples using Soft Q-learning.

Examples of server-side approaches include [17, 18]. In the case of [17], a server-side solution called Secure Web Application Proxy (SWAP) was introduced to detect and prevent XSS attacks. SWAP includes a reverse proxy that intercepts HTML responses and a modified Web browser to detect script content. Deployable transparently for clients, SWAP only requires a straightforward automated transformation of the original Web application. Experimental results demonstrated SWAP's effectiveness in correctly detecting exploits on authentic vulnerabilities in popular Web applications.

On the other hand, the paper [18] introduced a server-side automated framework called Cross-Site Scripting Secure Web Application Framework (XSS-SAFE) for detecting and mitigating XSS attacks in modern Web applications. The framework was evaluated on five real-world Java Server Pages (JSP) programs and demonstrated effective detection and mitigation of known and unknown XSS attacks with minimal false positives, no false negatives, and low runtime overhead.

Additionally, the method proposed in [19] seeks to detect XSS attacks by achieving load balance between clients and servers. The approach utilizes divergence measures to identify vulnerabilities and introduces an attribute clustering method, complemented by a rank aggregation technique, to effectively detect confounded JavaScripts.

According to [20] study, various detection techniques, such as supervised learning, unsupervised learning, reinforcement learning, deep learning, and metaheuristic algorithms, are examined to detect XSS attacks. As an instance, an ML-based model that can recognize the malicious attack vector before it is processed by the victim's system's browser

was introduced by Kaur Gurpreet et al. [21]. The study recognized blind XSS and cached XSS attacks using the linear support vector classification approach.

In their study [22], S. Sharma et al. emphasized the critical role of feature set extraction in web-based attack detection. To enhance detection accuracy, they introduced a feature set extraction approach, integrated with a machine learning (ML)-based intrusion detection model. The experiments were conducted using the Weka tool. The extracted data were fed into three ML models (J48, OneR, and Naïve Bayes) in Weka, with J48 demonstrating the most promising results among the classifiers.

In another study Wang et al. [23] proposed an approach to detect XSS worms in Online Social Network web pages. They differentiated benign and malicious web pages by analyzing the frequencies of scripting functions in both. To build their model, they utilized the ADTree decision tree and AdaBoost.M1 algorithms for classification, as ADTree demonstrated higher accuracy compared to other decision trees, and AdaBoost.M1 produced a strong classifier. The researchers also developed a feature extractor model to automatically capture features from the web pages, which played a crucial role in generating the classification model. For their experiments, benign and malicious samples were collected from DMOZ and XXSed Database. Four groups of features were extracted from web pages: keyword features, JavaScript features, HTML tag features, and URL features. Each of these features further covered sub-features, enhancing the precision and comprehensiveness of the XSS detection process.

Moreover, in their research Kascheev and Olenchikova [24] proposed supervised machine learning algorithms for detecting XSS attacks. Each request in the dataset was decoded into Unicode characters, and regular expressions were used to extract the query's parameters. The Word2Vec approach was used to extract the features. A comparison of four machine learning algorithms, including decision tree, Naïve Bayes classifier, logistic regression, and support vector machine, was conducted. The Decision Tree algorithm showed the most promising performance rates among them.

Similarly, in their study Banerjee et al. [25] explored the effectiveness of four machine learning algorithms, namely SVM, KNN, random forest, and logistic regression, in detecting XSS attacks. The logistic regression model was employed to map true and false values in the dataset. Among the four classifiers, the random forest Classifier demonstrated the most promising results, exhibiting high accuracy and a low false-positive rate.

Transitioning to unsupervised machine learning, a type that seeks to unveil patterns and structures within data without relying on explicit labeled training examples, stands in contrast to supervised learning. Unlike supervised learning, which relies on labeled data to learn patterns and make predictions, unsupervised learning solely relies on the inherent structure of the data itself. An illustration of this approach is found in the study [19]. The study introduced a new approach for detecting XSS attacks, emphasizing load balance between clients and servers. The method initiates vulnerability checking on the client side using divergence measures. If the suspicion level surpasses a predefined threshold, the request is discarded; otherwise, it is forwarded to the proxy for additional processing. To enhance detection, the proposed method integrated an attribute clustering technique with rank aggregation to identify confounded JavaScripts.

Reinforcement learning (RL) is a type of machine learning where an agent learns to make decisions by interacting with an environment. The agent takes actions to achieve specific goals and receives feedback in the form of rewards or penalties based on its actions. The main objective of RL is to learn an optimal policy that maps states to actions, maximizing the cumulative reward over time. For instance, in a study conducted by Fang et al. [26], the researchers proposed the RLXSS approach for XSS attack detection, employing reinforcement learning techniques. The authors introduced a novel concept that involves both adversarial and retraining models. The adversarial model is designed to counter adversarial attacks, while the retraining model is responsible for re-identifying malicious samples generated by the former model. To enhance the process, adversarial samples obtained from the adversarial model are utilized in the retraining model for optimization purposes. This approach aims to bolster the XSS detection system against adversarial threats using the principles of reinforcement learning. Furthermore, various researchers have employed genetic algorithms, hybrid methodologies, neural network models, deep learning techniques, and dynamic analysis-based approaches to enhance the accuracy of XSS attack detection. For example, the authors in [27] suggested and put into practice a new technique for lowering the false-positive rate in the XSS attack detection system that makes use of a modular deep neural network. The Word2vec technique was used to extract word associations from the large text corpus. The model was utilized in the detection and prevention of XSS attacks and consists of 50 features that were chosen using the Pearson correlation method. In another research [28], a platform with several criteria was presented for employing genetic algorithms (GA) to detect network intrusions. In the study, first, a pure GA with several selection techniques was used to build an intrusion detection system (IDS). Then, a new algorithm which combines the Tabu search (TS) and GA algorithms was proposed. Using data from the DARPA project, the suggested hybrid algorithm and pure GA were put to the test for detecting malicious traffic. The test results showed that the suggested hybrid algorithm outperforms the pure GA in terms of detection rate (DR) and detection accuracy. On the other hand, the authors Pavan Kumar et al. in their [29] research study used swarm-based intelligence

and deep learning approaches to identify website phishing assaults. The study's main goal is to employ the binary bat algorithm to discriminate between legitimate and fraudulent URLs and to increase speed using the Adam optimizer. In their experiment, 30 features are extracted for classification including URL length, IP address, user ID, iframe, port, redirect, mouseover, etc. Moreover, feature extraction is crucial in handling cybersecurity threats as it enables accurate, efficient, and adaptive detection of a wide range of cyber threats [30]. Consequently, researchers have recently dedicated significant attention to refining feature extraction methods in their investigations. A diverse range of feature extraction methods has been utilized to effectively represent data in a suitable form. For example, in [31] study, a new feature extraction method based on the auto-encoder structure was proposed to classify Android malware applications. Similarly [32, 33], introduced image-based feature extraction approaches. Another study [34] utilized a RoBERTa pretrained model to extract features from text to detect spam on social networks, improving the performance of the stacked BLSTM proposed network. Various transformer-based methods, such as BERT, distilBERT, Albert, and Electra, were also utilized in the same study [34] to extract features from text. In another investigation [35], deep contextualized Elmo embeddings were employed for feature extraction. Furthermore, for comparison purposes, other methods like word2vec, Glove, and fastText were employed in the experiments. In a similar context, the [36] research study employed BERT embeddings in the feature extraction phase of the experiment. These varied feature extraction methods empower cybersecurity analysts to effectively analyze and detect cyber threats across diverse data types and scenarios.

Feature extraction in XSS attack detection commonly involves two main methods: static analysis and dynamic analysis. These traditional methods have been extensively studied in the literature, as highlighted by Rodríguez et al. [37]. Static analysis involves examining the sanitized data without executing the script code to extract relevant features. In contrast, dynamic analysis concentrates on scrutinizing the behavior and data flow of the script during runtime. While both approaches have proven useful, they also come with their respective limitations. The static analysis method is limited by its inability to capture the dynamic behavior of the script, which may result in overlooking certain types of XSS attacks. On the other hand, dynamic analysis heavily relies on actual execution, which can lead to higher computational overhead and potential evasion by sophisticated attacks. The recognition of these limitations calls for further research and the exploration of more sophisticated and innovative feature extraction techniques.

By synthesizing the findings of these related works, we aim to build upon their collective insights to introduce a novel hybrid approach for XSS attack detection. Our proposed approach combines the strengths of the Universal Sentence Encoder and Word2Vec embeddings with advanced machine learning and deep learning techniques. This integrative approach aspires to offer a comprehensive and efficient solution for real-time XSS attack detection in web applications, augmenting the existing body of knowledge and contributing to the advancement of web security practices.

3 Methodology

3.1 The Proposed Model

In this section, the detailed framework of the proposed model for XSS attack detection is presented. The proposed approach encompasses three key stages. In the initial stage, a feature extraction process takes place, leveraging the Universal Sentence Encoder (USE) and Word2Vec to derive meaningful features from the dataset. The extracted features are then concatenated to create the final feature representation of the dataset. Following this, the dataset is partitioned into training and testing sets, with an 80–20% ratio, respectively. The subsequent stage involves the application of the seven machine learning classifiers introduced in this study. Finally, the proposed deep learning architecture is employed in the classification process. For detailed insights into the machine learning and deep learning architectures employed, additional insights are available in the subsequent subsections. Additionally, a visual representation of the proposed model is elucidated in Fig. 1.

3.2 Used Dataset

In this study, the Cross-Site Scripting (XSS) dataset for deep learning shared by SYED SAQLAIN HUSSAIN SHAH at Kaggle repository was utilized. Comprising 13,685 entries from PortSwigger and OWASP Cheat Sheets for XSS attacks, this dataset served as the foundation for our experiments and evaluation of XSS attack detection. The dataset provided a comprehensive collection of web content containing both benign samples and instances of XSS attacks. It encompassed a diverse range of XSS attack vectors, allowing us to assess the effectiveness of our models across different attack scenarios. By leveraging this dataset, we were able to conduct rigorous evaluations and draw meaningful conclusions regarding the performance and capabilities of our XSS attack detection methods.

3.3 Feature Extraction

Feature extraction plays a fundamental role in the XSS attack detection process. It involves transforming raw input data,

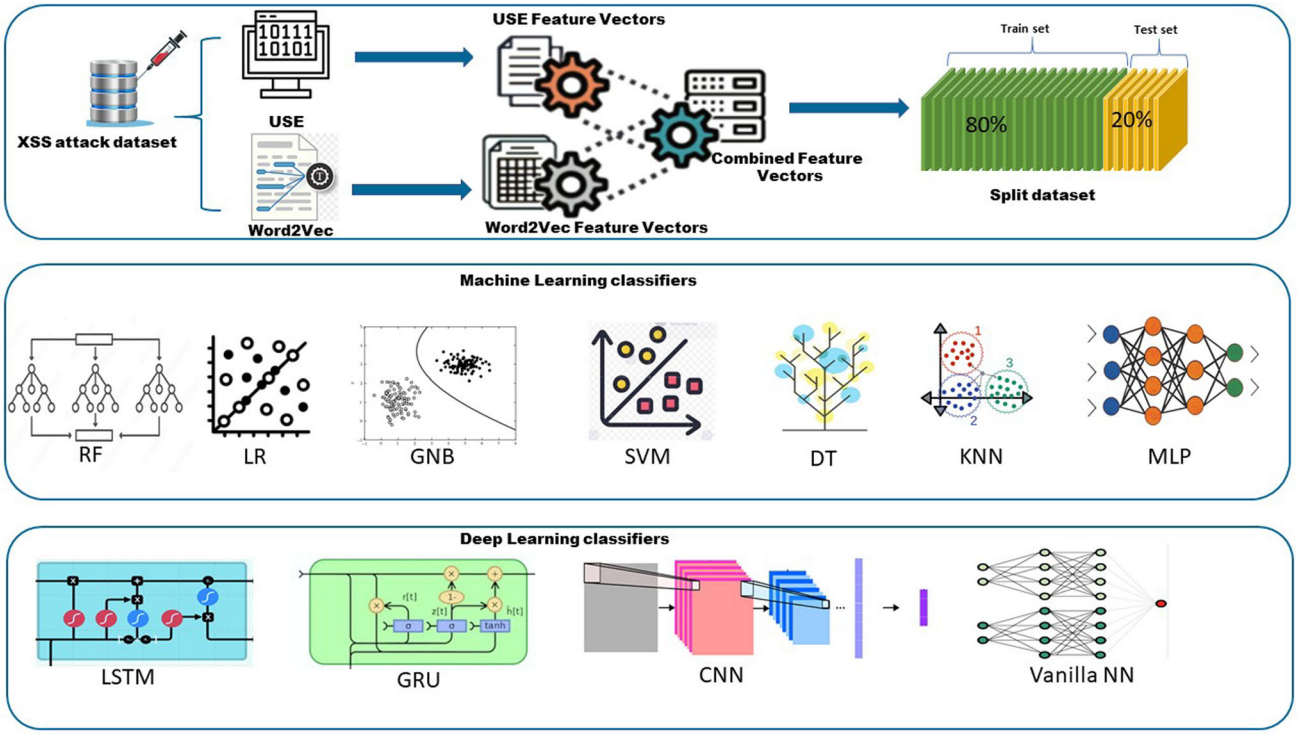

Fig. 1 The block diagram of the proposed method

such as web content, into a structured representation that can be effectively processed by machine learning algorithms. In this study, two feature extraction methods were explored: The Universal Sentence Encoder (USE) and the Word2vec approaches.

3.3.1 Word2vec

Word2Vec [38] is a popular word embedding technique that aims to represent words as dense vectors in a high-dimensional space. We employed the Word2Vec model to generate word-level embeddings for individual words present in the XSS attack payload. This technique allows us to capture semantic relationships between words based on their co-occurrence patterns in the training data.

In our implementation, a pretrained Word2Vec model trained on a large corpus of text data to ensure the availability of meaningful word embeddings was used. The Word2Vec model converts each word into a fixed-size vector of 100, where words with similar meanings are positioned closer to each other in the vector space.

To obtain word embeddings for the XSS attack payload, the text was tokenized into individual words, and each word's presence in the pretrained Word2Vec model's vocabulary was checked. If a word was found, its corresponding word embedding was extracted. For words not present in the model's vocabulary, they were either omitted or their embeddings were initialized as zero vectors.

By utilizing Word2Vec, the aim was to capture the semantic meanings of words within the XSS attack payload, enabling the model to discern critical patterns and contextual information. These word-level embeddings served as an essential component of the feature representation, complementing the sentence-level semantic understanding provided by the USE. The combination of these two techniques allowed for the creation of a comprehensive and powerful feature representation, enabling effective detection of XSS attacks.

3.3.2 Universal Sentence Encoder (USE)

The Universal Sentence Encoder is a versatile and powerful tool for natural language processing tasks. Developed by Google Research [11], the USE is designed to transform variable-length text inputs into fixed-dimensional vector representations, capturing the semantic meaning and contextual information of sentences. It utilizes a deep neural network architecture trained on a large corpus of text data, enabling it to generate high-quality embeddings that encode semantic relationships and similarities between sentences. The USE has demonstrated remarkable performance in a wide range of NLP applications, including text classification, semantic similarity, and sentiment analysis. In our study, we leverage

the capabilities of the USE as a feature extractor for XSS attack detection, harnessing its ability to capture meaningful representations of web content and enhance the accuracy of our detection models.

3.3.3 Unified Semantic Representation (USE-Word2Vec Hybrid Model)

The USE-Word2Vec Hybrid Model introduces a novel approach to enhance the feature extraction process for detecting XSS attacks. By leveraging both the Universal Sentence Encoder (USE) and Word2Vec embeddings, the objective is to create a comprehensive representation that captures both sentence-level and word-level semantic information.

In this hybrid model, the process begins by utilizing the USE to generate sentence-level embeddings for XSS attack payloads. The USE excels in capturing contextual and semantic meanings of entire sentences, providing a high-level understanding of the text. Simultaneously, Word2Vec is employed to generate word-level embeddings for individual words within the XSS attack payload, capturing fine-grained word meanings. To integrate these two types of embeddings, a concatenation technique is employed. Sentence-level embeddings from the USE are concatenated with word-level embeddings from Word2Vec for each XSS attack payload, resulting in a unified feature representation encompassing both sentence-level and word-level semantic information.

In the Python implementation, the Gensim library's Word2Vec pretrained module is utilized to train embeddings on the XSS dataset. The Word2Vec output vectors have a fixed size of 100. Subsequently, the USE is incorporated to generate sentence-level embeddings with a fixed dimensionality of 512. These embeddings are seamlessly concatenated with Word2Vec-derived word-level embeddings, resulting in a unified feature representation.

To evaluate the model's effectiveness, the dataset is divided into training and testing sets. The combined embeddings, with a size of 612, serve as input features for training machine learning models, offering a nuanced representation capturing the essence of XSS attack payloads at both sentence and word levels.

Mathematically, the USE embedding of a sentence s is denoted as $\mathrm{USE(s)} = \mathrm{fUSE(s)} \in \mathbb{R}^n$, where n is the dimensionality of USE embeddings. Similarly, the Word2Vec embedding of a word w is represented as $W2V(w) = fW2V(w) \in \mathbb{R}^m$, where m is the dimensionality of Word2Vec embeddings.

To create a unified feature representation, the USE embeddings of sentences are concatenated with the Word2Vec embeddings of the words in those sentences.

For a sentence s consisting of words w_1, w_2, …, w_k, the hybrid feature vector xs can be formulated as.

$x_s = [\mathrm{USE}(s), \mathrm{W2V}(w_1), \mathrm{W2V}(w_2), \ldots, \mathrm{W2V}(w_k)]$, resulting in a feature vector of dimensionality $n + k \times m$.

3.4 Classification Methods for XSS Attack Detection

3.4.1 Machine Learning (ML) techniques

Within the classification methods employed for XSS attack detection, machine learning techniques play a pivotal role. These techniques enable the development of predictive models that can automatically classify web content as either benign or potentially malicious with high accuracy. Machine learning algorithms learn from labeled training data, extracting patterns and relationships that can be utilized for effective classification. In our study, we leveraged various machine learning algorithms, including support vector machines (SVM), random forests, decision trees, logistic regression, k-nearest neighbor (KNN), and multilayer perceptron (MLP) to classify web content and detect XSS attacks. The mentioned algorithms were trained using feature vectors extracted from either the USE, word2vec, or the USE-Word2vec hybrid model. The hyperparameters of the proposed ML algorithms are illustrated in Table 1.

3.4.2 Deep Learning Techniques

Deep learning techniques have emerged as powerful tools for XSS attack detection, offering the ability to learn complex patterns and representations from data. These techniques, based on deep neural networks, are capable of automatically extracting hierarchical features and capturing intricate relationships within web content. In our study, we incorporated several simple deep learning architectures, such as convolutional neural networks (CNNs) and recurrent neural networks (RNNs), to classify web content and detect XSS attacks. These architectures were trained on feature representations obtained from the USE, Word2vec or from the USE-Word2vec hybrid model. By leveraging deep learning, we enhance the model's capacity to identify subtle indicators of XSS attacks, enabling more accurate and robust detection. The ability of deep learning models to learn from large-scale data and capture intricate patterns makes them valuable tools in the battle against XSS attacks and reinforces web application security. The intentional focus was on utilizing simple deep learning architectures to underscore the speed of XSS attack detection. By opting for simpler architectures, the goal was to streamline the computational processes involved in identifying and mitigating XSS attacks without compromising performance. These architectures were designed to strike a balance between accuracy and computational efficiency, allowing for swift detection of XSS attacks in real-time scenarios. While more complex deep learning

Table 1 Hyperparameters description

Classifier	Hyperparameter setting
RF	n_estimators: 100 max_depth: None min_samples_split: 2 min_samples_leaf: 1 max_features: 'auto'
DT	max_depth: None min_samples_split: 2 min_samples_leaf: 1
MLP	hidden_layer_sizes: 100 activation: relu solver: adam alpha: 0.0001
SVM	gamma:scale degree: 3
KNN	n_neighbors: 5 weights: uniform algorithm: auto leaf_size: 30 p: 2
LR	penalty: l2 C: 1.0 solver: lbfgs max_iter: 100
GRU	Loss: binary crossentropy Optimizer: adam Activation function: Sigmoid Learning rate:0.001 Epoch:100 Batch_size = 128
LSTM	Loss: binary crossentropy Optimizer: adam Activation function: Sigmoid Learning rate:0.001 Epoch:100 batch_size = 128
Vanilla 1	Optimizer: adam Learning rate:0.001 Epoch:100 Activation functions: Relu, Relu, Relu, Sigmoid Loss: binary crossentropy batch_size = 128
Vanilla 2	Loss: binary crossentropy Optimizer: Nadam Activation functions: Relu,Elu,Elu,Elu,Selu, Sigmoid Kernel initializer: uniform, glorot_normal, he_uniform, glorot_normal, he_normal, Dropout: 0.2 Epochs = 100
CNN	Filters in Convolution Layers:128,64,32 kernel_size:4 padding:same Activation functions: Relu, Relu, Relu, Sigmoid Learning rate:0.001 Epoch:100 Loss: binary crossentropy Optimizer: adam Batch_size = 128

architectures may offer higher predictive power, their computational demands could hinder the real-time responsiveness required for effective XSS attack detection. To this end, after conducting a series of experiments, the top-performing deep learning architectures were selected based on the most promising results. Through rigorous evaluation and comparison, five architectures were identified that outperformed others in terms of accuracy and performance. The chosen architectures include a simplified convolutional neural network (CNN), a basic long short-term memory (LSTM) neural network, a basic gated recurrent unit (GRU), and two vanilla neural network architectures. The first vanilla model comprises three fully connected layers, with 256, 128, and 64 neurons with relu activation function in each layer, respectively. The model also includes a fully connected output layer with the sigmoid activation function and binary cross-entropy as the loss function. The second vanilla model was derived from our different study, where hyperparameter tuning was conducted to address SQL injection attacks. In this study, we aim to assess the generalizability of the same model for detecting XSS attacks. The model consists of an input layer, five fully connected layers with 96, 192, 128, 128, and 32 neurons in each layer, respectively. The activation functions used are relu, elu, elu, elu, and selu, applied to each corresponding layer. Additionally, a dropout layer is included, followed by an output layer with the sigmoid activation function. The CNN model comprises three convolutional layers, with 128, 64, and 32 filters with relu activation function in each layer, respectively. Each convolutional layer is followed by a max pooling layer. A flatten layer is then applied, followed by a fully connected layer. The model concludes with an output layer utilizing the sigmoid activation function. The hyperparameters of the proposed deep learning architectures are shown in Table 1.

In our pursuit of simplified models to achieve optimal performance with swift detection, the LSTM model consists of a single LSTM layer with 20 units and relu activation function, followed by an output layer with the

sigmoid activation function. Similarly, the GRU models consist of a single GRU layer with 20 units and relu activation function, followed by an output layer with the sigmoid activation function. All models underwent training for 100 epochs using a batch size of 128.

4 Experimental Results

4.1 Evaluation Metrics

To assess the performance and effectiveness of our XSS attack detection models, a set of comprehensive evaluation metrics was employed. These metrics enabled us to measure the accuracy, precision, recall, and F1-score of our models, providing a holistic understanding of their performance. Additionally, the confusion matrix was utilized to visualize the distribution of true positive, true negative, false positive, and false negative predictions, aiding in identifying any biases or imbalances in our models' performance. Furthermore, to evaluate the models' classification performance across different thresholds, receiver operating characteristic (ROC) curves were employed, and the corresponding area under the curve (AUC) values were calculated. These metrics allowed us to assess the models' ability to balance true-positive rate and false-positive rate and provided insights into their overall performance. By utilizing this comprehensive set of evaluation metrics, the aim was to rigorously assess and compare the performance of our XSS attack detection models, enabling informed decisions about their effectiveness and suitability for real-world application scenarios.

4.2 Conducted Experiments

4.2.1 Experimental Setup

The experiments were conducted on a computational setup consisting of an 11th Gen Intel(R) Core (TM) i7-11700 processor running at 2.50 GHz, with 32 GB of RAM. The experiments were implemented using the Python programming language.

To evaluate the performance of our XSS attack detection models utilizing the Universal Sentence Encoder and Word2Vec, a comprehensive experiment was conducted. The experiment aimed to assess the accuracy and efficiency of both approaches in detecting XSS attacks. The XSS dataset, comprising a mixture of legitimate web content and malicious scripts commonly associated with XSS attacks, was utilized for the experiment. The dataset was randomly split into 80% training and 20% testing sets. The training set was used to train our models, while the testing set was reserved for evaluation purposes.

Various machine learning classifiers and deep learning architectures, including support vector machines (SVM), random forests, decision trees (DT), logistic regression (LR), k-nearest neighbor (KNN), and multilayer perceptron (MLP), were employed. Each model was trained on the training set using the respective feature extraction method.

To evaluate the performance of our models, a range of evaluation metrics was utilized. These metrics allowed us to assess the models' ability to correctly identify XSS attacks and legitimate content, while also quantifying any false positives or false negatives. Additionally, ROC curves were employed to analyze the models' performance across different classification thresholds. The AUC values were calculated to determine the overall discrimination ability of each model.

Finally, to ensure a thorough understanding of our evaluation methodology, the hyperparameters utilized in the proposed machine learning classifiers were detailed. These hyperparameters play a crucial role in the training and performance of the models. Subsequently, the hyperparameters of the selected deep learning architectures were delineated to provide a holistic view of our experimental setup. The hyperparameters of the proposed machine learning and deep learning architectures are shown in Table 1.

4.2.2 Obtained Results

Machine learning results Seven machine learning classifiers, as mentioned before, have been used in the experiments. Tables 2, 3, and 4 present the results obtained when using the Universal Sentence Encoder (USE), Word2Vec, and the USE-Word2vec approaches as feature extractors, respectively. Additionally, Fig. 2 illustrates the ROC curves of ML classifiers using the USE-Word2vec hybrid method for XSS attack detection.

Deep Learning Results In our study, in order to achieve optimal results, various combinations of simple deep neural network architectures were employed.

The obtained results from the previously mentioned architectures are shown in Tables 5, 6, and 7.

Time Study Given the paramount importance of time in our study, a thorough time analysis was conducted to assess the efficiency of XSS attack detection. Tables 8, 9, and 10 display the results of our investigation, focusing on the training time required for machine learning. Moreover, Tables 11, 12, and 13 present the training time results for deep learning using various feature extraction approaches. Additionally, Table 14 illustrates the detection time needed for the proposed approach in detecting XSS attacks.

Table 2 Machine learning results with universal sentence encoder

Classifier	Accuracy%	F1-score	Recall	Precision	FP	FN	ROC-AUC score
LR	96.06	0.9645	0.9454	0.9844	23	84	0.9906
SVM	98.39	0.9853	0.9742	0.9966	5	39	0.9965
Gaussian NB	90.17	0.9125	0.8790	0.9486	76	193	0.9298
DT	94.81	0.9515	0.9601	0.9432	84	58	0.9485
KNN	97.92	0.9809	0.9702	0.9919	12	45	0.9949
MLP	**98.61**	0.9871	0.9885	0.9858	21	17	0.9980
RF	98.10	0.9826	0.9703	0.9953	7	45	0.9962

The highest result for the accuracy metric is highlighted in bold

Table 3 Machine learning results with word2vec

Classifier	Accuracy%	F1-score	Recall	Precision	FP	FN	ROC-AUC score
LR	95.11	0.9564	0.9211	0.9946	8	126	0.9921
SVM	95.00	0.9554	0.9198	0.9939	9	128	0.9907
Gaussian NB	91.75	0.9284	0.8731	0.9912	13	213	0.9315
DT	97.48	0.9771	0.9584	0.9966	5	64	0.9957
KNN	97.26	0.9751	0.9576	0.9932	10	65	0.9945
MLP	97.04	0.9731	0.9556	0.9912	13	68	0.9965
RF	**97.55**	0.9778	0.9584	0.9980	3	64	0.9977

The highest result for the accuracy metric is highlighted in bold

Table 4 Machine learning results with USE-Word2vec hybrid model

Classifier	Accuracy%	F1-score	Recall	Precision	FP	FN	ROC-AUC score
LR	97.48	0.9770	0.9644	0.9899	15	54	0.9921
SVM	97.99	0.9817	0.9678	0.9959	6	49	0.9907
Gaussian NB	93.54	0.9427	0.9043	0.9844	23	154	0.9316
DT	98.25	0.9837	0.9877	0.9797	30	18	0.9957
KNN	98.58	0.9868	0.9845	0.9892	16	23	0.9945
MLP	99.10	0.9915	0.9932	0.9899	15	10	0.9965
RF	**99.45**	0.9949	0.9926	0.9973	4	11	0.9978

The highest result for the accuracy metric is highlighted in bold

5 Discussion

In the analysis of the machine learning results based on the obtained data from Tables 2, 3, and 4, it was observed that the multilayer perceptron (MLP) classifier exhibited superior performance when utilizing the Universal Sentence Encoder (USE) as the feature extraction technique. The MLP classifier demonstrated higher accuracy, precision, and recall compared to other machine learning classifiers in this configuration. Conversely, when employing Word2Vec embeddings and the USE-Word2Vec hybrid model, the random forest (RF) classifier demonstrated remarkable performance, outperforming other classifiers in terms of accuracy, precision, recall, and other metrics with good false positive and false negative rates. This indicates the effectiveness of the RF classifier in leveraging the word-level representations captured by Word2Vec in combination with the semantic understanding provided by USE. Remarkably, the USE-Word2Vec hybrid model surpassed all previous models in our evaluation, achieving superior results across all evaluation metrics. The fusion of sentence-level semantics from USE and fine-grained word meanings from Word2Vec in the hybrid model led to a comprehensive and powerful feature representation. This enhanced representation facilitated the model's ability to discern complex patterns and contextual information, resulting in improved detection performance for XSS attacks. Overall, our findings demonstrate the importance of tailoring the choice of classifier to the specific feature extraction technique. While MLP excelled with USE

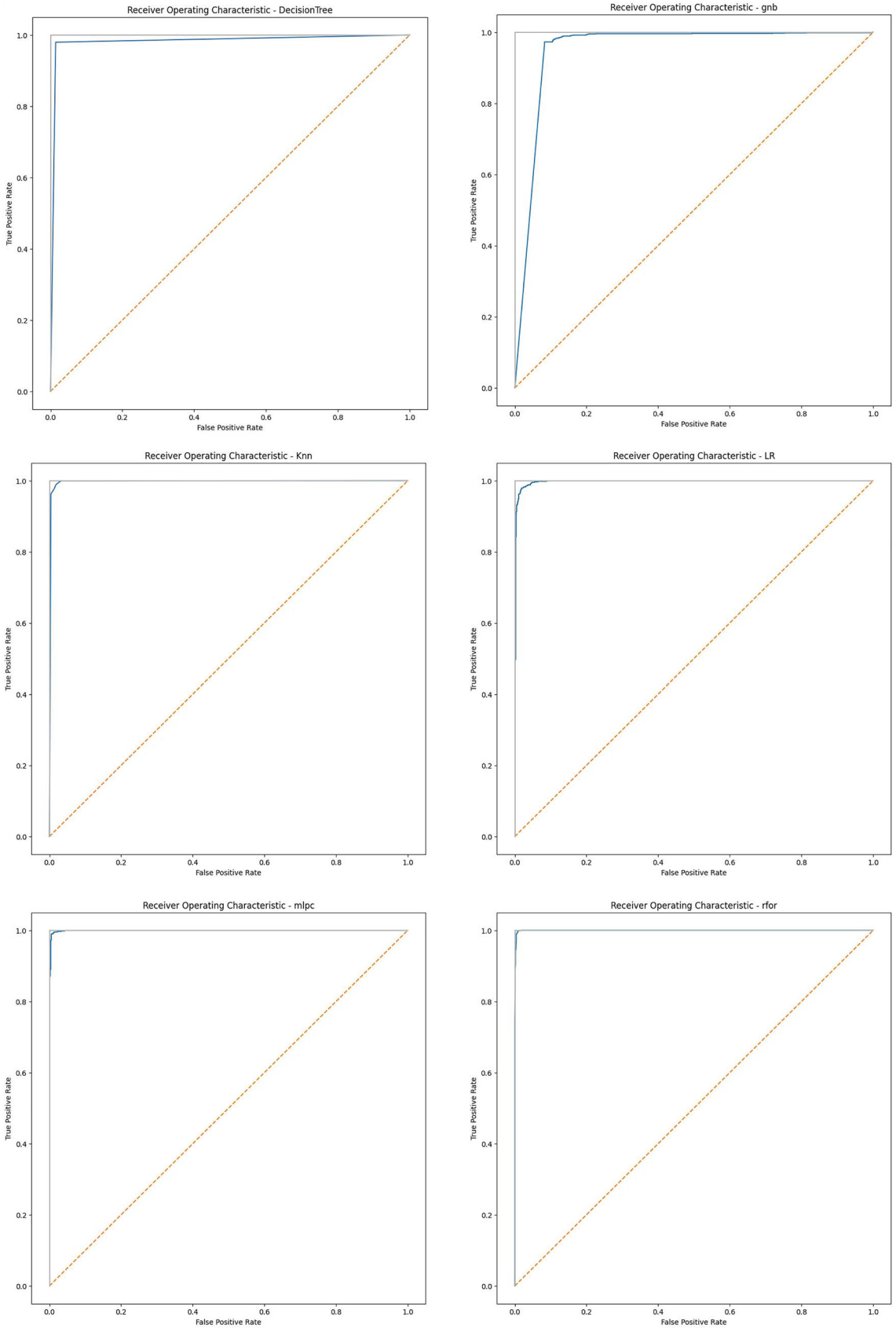

Fig. 2 ROC curves of ML classifiers using USE-Word2Vec hybrid method for xss attack detection

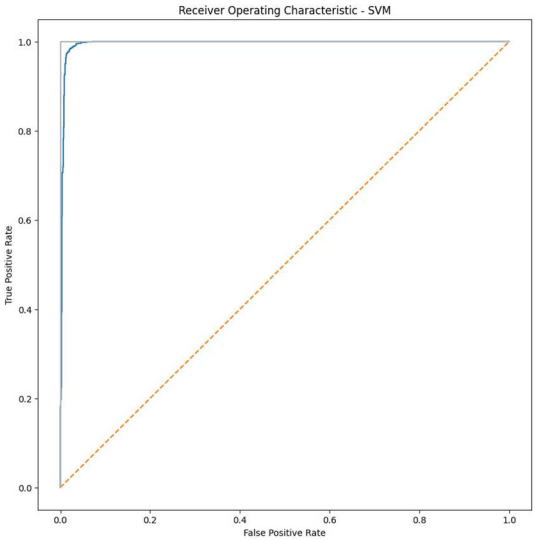

Fig. 2 continued

embeddings, RF proved more effective with Word2Vec and the USE-Word2Vec hybrid model. Moreover, the promising results achieved by the hybrid model emphasize the potential of integrating diverse natural language processing techniques to enhance the performance of XSS attack detection systems.

In our exploration of deep learning architectures, as can be seen in Tables 5, 6, and 7, our findings revealed notable distinctions in performance among these deep learning architectures when coupled with different feature extraction techniques. Notably, the second vanilla model, fine-tuned to address SQL injection attacks in a prior study, demonstrated superior performance when the USE and the USE-Word2Vec hybrid model were used as feature extractors. Conversely, the CNN architecture outperformed the other models when Word2Vec embeddings were employed. These results underscore the importance of tailoring deep learning architectures to specific feature extraction methods, highlighting the value of model customization for varying data domains. The remarkable performance of the second vanilla model when combined with USE and the hybrid model highlights the potential of leveraging hyperparameter tuning for broader applicability across different security challenges. Additionally, the superiority of the CNN architecture with Word2Vec emphasizes the significance of aligning architecture choices with the characteristics of the underlying data representation. This investigation provides valuable insights into the optimal

Table 5 Deep learning results with USE

DL model	Accuracy%	F1-score	Recall	Precision	FP	FN
Vanilla NN 1	98.94	0.9902	0.9885	0.9919	12	17
Vanilla NN 2	**98.98**	0.9905	0.9885	0.9926	11	17
LSTM	97.92	0.9808	0.9778	0.9838	24	33
GRU	98.10	0.9825	0.9792	0.9858	21	31
CNN	98.08	0.9902	0.9885	0.9919	12	17

The highest result for the accuracy metric is highlighted in bold

Table 6 Deep learning results with word2vec

DL model	Accuracy%	F1-score	Recall	Precision	FP	FN
Vanilla NN 1	96.64	0.9683	0.9880	0.9493	75	17
Vanilla NN 2	97.26	0.9751	0.9558	0.9953	7	68
LSTM	93.94	0.9435	0.9499	0.9371	93	73
GRU	93.61	0.9437	0.8999	0.9919	12	163
CNN	**97.66**	0.9788	0.9603	0.9980	3	61

The highest result for the accuracy metric is highlighted in bold

Table 7 Deep learning results with USE-Word2Vec Hybrid Model

DL model	Accuracy%	F1-score	Recall	Precision	FP	FN
Vanilla NN 1	99.12	0.9919	0.9939	0.9899	15	9
Vanilla NN 2	**99.16**	0.9922	0.9899	0.9946	8	15
LSTM	98.47	0.9858	0.9838	0.9878	18	24
GRU	94.49	0.9497	0.9351	0.9648	52	99
CNN	99.05	0.9912	0.9946	0.9878	18	8

The highest result for the accuracy metric is highlighted in bold

Table 8 Time study results for USE with ML classifiers

Classifier	Time (sec)	Time/sample (ms)
LR	0.17	0.016
SVM	15.8	1.443
Gaussian NB	**0.003**	0.00027
DT	13.77	1.258
KNN	18.5	1.69
MLP	0.029	0.003
RF	27.9	2.548

The minimum time result is highlighted in bold

Table 9 Time study results for Word2vec with ML classifiers

Classifier	Time (sec)	Time/sample (ms)
LR	0.090	0.0008
SVM	3.857	0.0352
Gaussian NB	0.012	0.0001
DT	0.3008	0.0027
KNN	**0.0012**	0.0000
MLP	7.128	0.0651
RF	2.625	0.0240

The minimum time result is highlighted in bold

Table 10 Time study results for USE-Word2vec hybrid model with ML classifiers

Classifier	Time (sec)	Time/sample (ms)
LR	0.39	0.035
SVM	14.38	1.31
Gaussian NB	0.052	0.005
DT	7.4	0.68
KNN	**0.0069**	0.00063
MLP	10.93	0.998
RF	19.33	1.76

The minimum time result is highlighted in bold

Table 11 Time study results for USE with DL models

DL model	Time (sec)	Time/sample (ms)
Vanilla NN 1	31.3	0.31
Vanilla NN 2	47.22	0.431
LSTM	**10.45**	**0.095**
GRU	498.80	4.57
CNN	87.91	0.8

The minimum time result is highlighted in bold

Table 12 Time study results for Word2vec with DL models

DL model	Time (sec)	Time/sample (ms)
Vanilla NN 1	10.68	1.084
Vanilla NN 2	**0.171**	**0.017**
LSTM	9.466	0.961
GRU	30.17	3.062
CNN	96.04	9.747

The minimum time result is highlighted in bold

Table 13 Time results for USE-Word2Vec Hybrid Model with DL models

DL model	Time (sec)	Time/sample (ms)
Vanilla NN 1	40.5	3.6
Vanilla NN 2	57.9	5.2
LSTM	**17.3**	**1.5**
GRU	910	83
CNN	141.9	12.9

The minimum time result is highlighted in bold

combinations of deep learning models and feature extraction techniques for robust and accurate XSS attack detection.

Moving to Fig. 2 which shows the receiver operating characteristic (ROC) curves of ML classifiers when the hybrid model is utilized as a feature extractor. As known the ROC curves are used to provide valuable insights into the performance of our XSS attack detection models. The ROC curves illustrate the trade-off between the true-positive rate (sensitivity) and the false-positive rate (1—specificity) at various classification thresholds. A comprehensive analysis of the ROC curves allows us to evaluate the models' ability to discriminate between positive (XSS attack) and negative (non-XSS attack) instances across different threshold settings. Throughout our analysis, we observed that the USE-Word2Vec hybrid model consistently outperformed all other models, exhibiting ROC curves with higher AUC values. This finding indicates the hybrid model's ability to effectively separate XSS attack instances from non-attack instances, resulting in improved detection performance. Moreover, when we utilized the USE as the feature extractor, the MLP classifier demonstrated excellent performance, as evidenced by its high AUC value on the ROC curve. The MLP classifier's ability to leverage the semantic understanding of sentences provided by USE likely contributed to its strong performance in this configuration. In contrast, when employing Word2Vec embeddings, the RF classifier achieved remarkable results, indicated by its high AUC value on the corresponding ROC

Table 14 Detection time for the proposed approach

Model	Time (sec)	Time/sample (ms)
LR	0.0024	0.0009
SVM	0.6259	0.23
Gaussian NB	0.0224	0.008
DT	0.0039	0.001
KNN	0.1925	0.070
MLP	0.0050	0.002
RF	0.0319	0.012
Vanilla NN 1	0.3546	0.129
Vanilla NN 2	0.2154	0.079
LSTM	0.1697	0.062
GRU	2.3798	0.869
CNN	0.2786	0.101

curve. The RF classifier's proficiency in capturing word-level semantic relationships contributed to its superiority with Word2Vec feature extraction.

Our analysis of the ROC curves reaffirms the efficacy of the USE-Word2Vec hybrid model in detecting XSS attacks, showcasing its ability to leverage both sentence-level and word-level semantic information. The ROC curves also highlight the importance of selecting appropriate classifiers based on the chosen feature extraction method, underscoring the significance of a tailored approach in achieving optimal performance.

Upon examining the time study results, a notable pattern emerged. When the Universal Sentence Encoder was employed as the feature extraction method, the Gaussian Naive Bayes classifier exhibited the shortest training time among all machine learning models. This finding suggests that the simplicity and efficiency of the Gaussian NB classifier allowed for swift processing and model training when leveraging the semantic understanding offered by USE. Conversely, when Word2Vec embeddings and the USE-Word2Vec hybrid model were utilized as feature extractors, the KNN classifier demonstrated the least training time. The KNN classifier's time efficiency can be attributed to its simple yet effective mechanism of finding the k-nearest neighbors based on word-level semantic relationships captured by Word2Vec. Similarly, the hybrid model's ability to leverage both USE and Word2Vec embeddings likely contributed to the KNN classifier's enhanced efficiency in this configuration. On the other hand, in our time study of deep learning models, we investigated the training times for different architectures, aiming to assess their efficiency in XSS attack detection with varying feature extraction methods. The results, as presented in Tables 11, 12, and 13, highlight intriguing patterns regarding the training times of the LSTM architecture and the second vanilla model. When the USE and the hybrid feature extraction methods were utilized, the LSTM architecture demonstrated the shortest training time among all deep learning models. This outcome suggests that the LSTM's architecture, with its recurrent connections and memory cells, efficiently harnessed the semantic understanding and contextual information provided by USE and the hybrid model. This contributed to the LSTM's faster convergence during the training process. In contrast, when Word2Vec was employed as the feature extraction method, the second Vanilla model showcased the shortest training time. The second Vanilla model's architecture effectively adapted to the word-level semantic representations captured by Word2Vec. Consequently, the second Vanilla model demonstrated remarkable efficiency in this context, indicating the significance of tailored model configurations for specific feature extraction techniques. These results shed light on the importance of selecting suitable classifiers based on the chosen feature extraction technique. While the Gaussian NB classifier proved advantageous with USE, the KNN classifier emerged as the optimal choice with Word2Vec and the hybrid model. These insights can significantly impact real-world application scenarios, where time-critical XSS attack detection is essential for ensuring web application security. Indeed, it is noteworthy to emphasize the impressive training times achieved by all the proposed models in our study. Across both machine learning classifiers and deep learning architectures, the training times remain remarkably short, underscoring the efficiency of our approach for XSS attack detection. For the machine learning classifiers, even in the worst-case scenarios, the training time does not exceed 3 ms per sample. This rapid training duration indicates that the proposed models can process large datasets efficiently and are well-suited for real-time or time-sensitive applications. The ability to achieve such fast training times enhances the feasibility of deploying these models in web security systems, where swift and accurate detection of XSS attacks is paramount. Similarly, in the case of deep learning models, the training times remain impressively low, not surpassing 12 ms per sample. This demonstrates the effectiveness of our chosen deep learning architectures and their ability to efficiently leverage the feature extraction methods. Such swift training times for deep learning models further reinforce the practical applicability of our approach in real-world web application environments, where responsiveness and accuracy are crucial for maintaining web security. The detection time of the hybrid proposed model is undoubtedly a pivotal aspect and one of the most significant strengths of our approach. Even in worst-case scenarios, the detection time for the hybrid model remains exceptionally swift, not exceeding 1 ms per sample. This remarkable speed in detecting XSS attacks is of paramount importance in real-world web security applications, where timely response and rapid identification of

threats are critical. The ability to achieve such fast detection times with the hybrid model is a testament to the effectiveness of combining the USE and Word2Vec embeddings. The hybrid model's ability to harness the strengths of both feature extraction techniques results in a comprehensive and efficient representation of the input data, enabling precise and rapid detection of XSS attacks. With detection times below 1 ms per sample, the hybrid model surpasses expectations in terms of real-time responsiveness. This makes it well-suited for deployment in high-traffic web environments, where quick and accurate XSS attack detection is essential for maintaining web application security. The hybrid model's impressive performance is a testament to its ability to effectively process large volumes of web traffic without compromising on accuracy or speed. The combination of swift training times and detection times underlines the overall efficiency and practical applicability of our proposed approach. It offers a compelling solution for XSS attack detection that can be seamlessly integrated into web security systems, enhancing protection against malicious attacks without introducing unnecessary delays or computational overhead.

6 Conclusion

The role of speed in detecting XSS attacks is crucial for effectively mitigate their impact, prevent unauthorized access or data breaches, and safeguard the integrity of web applications. Real-time monitoring and detection mechanisms, combined with efficient algorithms and feature extraction techniques, are essential to enable swift identification and response to XSS threats. By prioritizing speed in XSS attack detection, organizations can significantly enhance their ability to protect web applications and users from the ever-present threat of XSS attacks. In this study, a comprehensive exploration of XSS attack detection has been embarked upon, leveraging a unique hybrid approach that combines the USE and Word2Vec embeddings. The objective has been to develop an efficient and accurate XSS attack detection system that addresses the dual challenges of feature extraction and model selection. Through a series of extensive experiments, the efficacy of the proposed hybrid model has been demonstrated, seamlessly integrating the semantic understanding of sentences from USE with the fine-grained word-level representations captured by Word2Vec. The results showcased superior performance across various evaluation metrics, underscoring its unmatched accuracy and efficiency in detecting XSS attacks. Investigation into a diverse range of machine learning and deep learning architectures revealed intriguing patterns in model performance, emphasizing the importance of selecting the appropriate model for a given feature extraction method to optimize detection outcomes. In terms of training and detection times, the proposed models surpassed expectations, with training times not exceeding 3 ms per sample in machine learning classifiers and 12 ms per sample in deep learning architectures. Detection time remained impressively short, not surpassing 1 ms per sample even in worst-case scenarios, ensuring real-time responsiveness crucial for web security in dynamic and high-traffic environments.

While the proposed hybrid model demonstrated promising results in detecting XSS attacks, several limitations warrant consideration. Foremost among these is the necessity to assess the model's generalizability across diverse web application domains and attack scenarios. Although the experiments yielded positive outcomes, the model's performance might fluctuate when confronted with datasets from varying contexts or encountering novel attack patterns absent in the training data. To mitigate this limitation, future research endeavors should prioritize training the models using comprehensive datasets representing a spectrum of web application domains and encompassing diverse attack vectors. By exposing the model to a broader range of scenarios, its adaptability and robust performance across real-world environments can be enhanced. Additionally, ongoing monitoring and updating of the model with new data and emerging attack patterns are imperative to sustain its efficacy and relevance over time. Future research can explore the integration of additional natural language processing techniques and alternative deep learning architectures to further enhance the hybrid model's capabilities. Moreover, expanding the dataset and testing the approach across diverse web application domains will provide valuable insights into its generalizability and adaptability.

Acknowledgements The authors would like to thank SYED SAQLAIN HUSSAIN SHAH for sharing dataset.

Funding Open access funding provided by the Scientific and Technological Research Council of Türkiye (TÜBİTAK). No specific grants from funding agencies in the public, commercial, or not-for-profit sectors were received for this research.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

Content of the Publication During the preparation of this work, the author(s) utilized AI-assisted services to enhance the language and readability of the manuscript. This step was taken to ensure the clarity and coherence of the content, ultimately improving the overall quality of the paper. After using this service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

References

1. Hannousse, A.; Yahiouche, S.; Nait-Hamoud, M.C.: Twenty-two years since revealing cross-site scripting attacks: a systematic mapping and a comprehensive survey. (2022). https://arxiv.org/abs/2205.08425.
2. Sharif MHU.: Web attacks analysis and mitigation techniques. Int. J. Eng. Res. Technol. 10–2 (2022)
3. Gupta, B.B.; Chaudhary, P.: Cross-site scripting attacks: classification, attack, and countermeasures. CRC Press, Boca Raton (2020)
4. Li, X.; Xue, Y.: A survey on server-side approaches to securing web applications. ACM Comput. Surv. (CSUR) **46**, 1–29 (2014)
5. Bakour, K.; Ünver, H.M.; Ghanem, R.: A deep camouflage: evaluating android's anti-malware systems robustness against hybridization of obfuscation techniques with injection attacks. Arab. J. Sci. Eng. **44**, 9333–9347 (2019)
6. Rathore, S.; Sharma, P.K.; Park, J.H.: XSSClassifier: an efficient XSS attack detection approach based on machine learning classifier on SNSs. J. Inform. Process. Syst. (2017). https://doi.org/10.3745/JIPS.03.0079
7. Chen, X.; Li, M.; Jiang, Y.; Sun, Y.: A comparison of machine learning algorithms for detecting XSS attacks. In: Artificial intelligence and security: 5th international conference, ICAIS 2019, New York, NY, USA, July 26–28, 2019, Proceedings, Part IV 5, pp. 214–24 Springer (2019).
8. Melicher, W.; Fung, C.; Bauer, L.; Jia, L.: Towards a lightweight, hybrid approach for detecting DOM XSS vulnerabilities with machine learning. Proc. Web Conf. **2021**, 2684–2695 (2021)
9. Fang, Y.; Li, Y.; Liu, L.; Huang, C.: DeepXSS: cross site scripting detection based on deep learning. In: Proceedings of the 2018 international conference on computing and artificial intelligence, pp. 47–51 (2018).
10. Maurel, H.; Vidal, S.; Rezk, T.: Statically identifying XSS using deep learning. Sci. Comput. Program. **219**, 102810 (2022)
11. Cer, D.; Yang, Y.; Kong, S.; Hua, N.; Limtiaco, N.; John, R.S.; et al.: Universal sentence encoder. (2018). https://arxiv.org/abs/1803.11175.
12. Thajeel, I.K.T.; Samsudin, K.; Hashim, S.J.; Hashim, F.: Machine and deep learning-based xss detection approaches: a systematic literature review. J. King Saud Univ. Comput. Inform. Sci. **35**, 101628 (2023)
13. Kirda, E.; Kruegel, C.; Vigna, G.; Jovanovic, N.: Noxes: a client-side solution for mitigating cross-site scripting attacks. In: Proceedings of the 2006 ACM symposium on Applied computing. pp. 330–7 (2006).
14. Abikoye, O.C.; Abubakar, A.; Dokoro, A.H.; Akande, O.N.; Kayode, A.A.: A novel technique to prevent SQL injection and cross-site scripting attacks using Knuth-Morris-Pratt string match algorithm. EURASIP J. Inf. Secur. **2020**, 1–14 (2020)
15. Zhou, Y.; Wang, P.: An ensemble learning approach for XSS attack detection with domain knowledge and threat intelligence. Comput. Secur. **82**, 261–269 (2019)
16. Wang, Q.; Yang, H.; Wu, G.; Choo, K.-K.R.; Zhang, Z.; Miao, G., et al.: Black-box adversarial attacks on XSS attack detection model. Comput Secur. **113**, 102554 (2022)
17. Wurzinger, P.; Platzer, C.; Ludl, C.; Kirda, E.; Kruegel, C.: SWAP: mitigating XSS attacks using a reverse proxy. In: 2009 ICSE Workshop on Software Engineering for Secure Systems, pp. 33–9. IEEE (2009).
18. Gupta, S.; Gupta, B.B.: XSS-SAFE: a server-side approach to detect and mitigate cross-site scripting (XSS) attacks in JavaScript code. Arab. J. Sci. Eng. **41**, 897–920 (2016)
19. Goswami, S.; Hoque, N.; Bhattacharyya, D.K.; Kalita, J.: An unsupervised method for detection of XSS attack. Int. J. Netw. Secur. **19**, 761–775 (2017)
20. Kaur, J.; Garg, U.; Bathla, G.: Detection of cross-site scripting (XSS) attacks using machine learning techniques: a review. Artif. Intell. Rev. **56**, 12725–12769 (2023)
21. Kaur, G.; Malik, Y.; Samuel, H.; Jaafar, F.: Detecting blind cross-site scripting attacks using machine learning. In: Proceedings of the 2018 international conference on signal processing and machine learning, pp. 22–5 (2018).
22. Sharma, S.; Zavarsky, P.; Butakov, S.: Machine learning based intrusion detection system for web-based attacks. In: 2020 IEEE 6th intl conference on big data security on cloud (BigDataSecurity), IEEE Intl conference on high performance and smart computing, (HPSC) and IEEE Intl conference on intelligent data and security (IDS), pp. 227–30. IEEE (2020).
23. Wang, R.; Jia, X.; Li, Q.; Zhang, S.: Machine learning based cross-site scripting detection in online social network. In: 2014 IEEE Intl Conf on high performance computing and communications, 2014 ieee 6th intl symp on cyberspace safety and security, 2014 IEEE 11th Intl Conf on Embedded Software and Syst (HPCC, CSS, ICESS), pp. 823–826. IEEE (2014).
24. Kascheev, S.; Olenchikova, T.: The detecting cross-site scripting (xss) using machine learning methods. In: 2020 global smart industry conference (GloSIC), pp. 265–70. IEEE (2020).
25. Banerjee, R.; Baksi, A.; Singh, N.; Bishnu, S.K.: Detection of XSS in web applications using Machine Learning Classifiers. In: 2020 4th international conference on electronics, materials engineering & nano-technology (IEMENTech), pp. 1–5. IEEE (2020).
26. Fang, Y.; Huang, C.; Xu, Y.; Li, Y.: RLXSS: Optimizing XSS detection model to defend against adversarial attacks based on reinforcement learning. Future Internet **11**, 177 (2019)
27. Alqarni, A.A.; Alsharif, N.; Khan, N.A.; Georgieva, L.; Pardade, E.; Alzahrani, M.Y.: MNN-XSS: modular neural network based approach for XSS attack detection. Comput. Mater. Cont. **70**, 4075–4085 (2022)
28. Bakour, K.; Daş, G.S.; Ünver, H.M.: An intrusion detection system based on a hybrid Tabu-genetic algorithm. In: 2017 international conference on computer science and engineering (UBMK), pp. 215–20. IEEE (2017).
29. Kumar, P.P.; Jaya, T.; Rajendran, V.: SI-BBA–a novel phishing website detection based on Swarm intelligence with deep learning. Mater. Today Proc. **80**, 3129–3139 (2023)
30. Doğan, E.; BAKIR, H.: Hiperparemetreleri Ayarlanmış Makine Öğrenmesi Yöntemleri Kullanılarak Ağdaki Saldırıların Tespiti. In: International conference on pioneer and innovative studies, pp. 274–86 (2023)
31. Bakır, H.; Bakır, R.: DroidEncoder: malware detection using autoencoder based feature extractor and machine learning algorithms. Comput. Electr. Eng. **110**, 108804 (2023)
32. Ünver, H.M.; Bakour, K.: Android malware detection based on image-based features and machine learning techniques. SN Appl. Sci. **2**, 1–15 (2020)

33. Bakour, K.; Ünver, H.M.: DeepVisDroid: android malware detection by hybridizing image-based features with deep learning techniques. Neural Comput. Appl. **33**, 11499–11516 (2021)
34. Ghanem, R.; Erbay, H.; Bakour, K.: Contents-based spam detection on social networks using RoBERTa embedding and stacked BLSTM. SN Comput. Sci. **4**, 380 (2023)
35. Ghanem, R.; Erbay, H.: Spam detection on social networks using deep contextualized word representation. Multimed. Tools Appl. **82**, 3697–3712 (2023)
36. Ghanem, R.; Erbay, H.: Context-dependent model for spam detection on social networks. SN Appl. Sci. **2**, 1–8 (2020)
37. Rodríguez, G.E.; Torres, J.G.; Flores, P.; Benavides, D.E.: Cross-site scripting (XSS) attacks and mitigation: a survey. Comput. Netw. **166**, 106960 (2020)
38. Mikolov, T.; Chen, K.; Corrado, G.; Dean, J.: Efficient estimation of word representations in vector space. (2013). https://arxiv.org/abs/1301.3781.